

Athens Sports SAAS Development Plan
Technology Stack Decision
Based on your requirements:

Backend: Flask (Python) with REST API

Frontend: React (JavaScript/JSX)

Database: PostgreSQL (for production), SQLite (for initial development)

Authentication: JWT-based with Flask-JWT-Extended

Deployment: Separate services (Flask API + React frontend)

Development Philosophy

We'll build this incrementally with working software at each step:

Start with minimal working system

Add features one by one

Test thoroughly at each stage

Refactor when needed

PHASE 1: FOUNDATION (Week 1)

Step 1.1: Project Setup & Basic Flask API

Initialize Flask project structure

Set up virtual environment

Create basic Flask app with health check endpoint

Configure SQLite database

Create basic User model

Set up Flask-Migrate for database migrations

Deliverable: Working Flask server with /api/health endpoint

Step 1.2: React Frontend Setup

Create React app with Create React App

Remove TypeScript configuration

Set up basic folder structure

Create simple home page

Connect to Flask backend with axios

Deliverable: React app displaying "Athens Sports" connected to Flask

Step 1.3: Authentication System

Backend:

Install Flask-JWT-Extended

Create User registration endpoint (/api/auth/register)

Create User login endpoint (/api/auth/login)

Create protected route example

Frontend:

Create Login component

Create Signup component

Store JWT token in localStorage

Add axios interceptor for auth headers

Deliverable: User can register and login

PHASE 2: CORE MODELS & ATHLETE MANAGEMENT (Week 2)

Step 2.1: Database Schema & Models

Create models:

User (extends auth)

Athlete (linked to User)

Team

Coach (linked to User)

Backend:

Create model definitions

Set up relationships (Team→Coach→Athlete)

Create migration

Add seed data function

Deliverable: Database with User, Athlete, Team models

Step 2.2: Athlete Management (Coach View)

Backend:

Create /api/athletes endpoints (GET, POST)

Add coach-only middleware

Add athlete filtering by team

Frontend:

Create AthleteList component

Create AddAthlete form (simple modal)

Display athlete cards with basic info

Deliverable: Coach can view and add athletes to their team

PHASE 3: WELLNESS TRACKING (Week 3)

Step 3.1: Wellness Data Model

Backend:

Create WellnessEntry model

Add fields: sleep, stress, soreness, nutrition, mood, readinessScore

Create /api/wellness endpoints

Frontend:

Create DailyWellnessForm component

Add form with sliders for each metric

Display simple wellness history chart

Deliverable: Athletes can submit daily wellness check

Step 3.2: Athlete Dashboard - Wellness View

Frontend:

Create AthleteDashboard component

Display wellness trends (Chart.js)

Show readiness scores over time

Add date filtering

Backend:

Add wellness statistics endpoint

Add date range filtering

Deliverable: Athlete can see their wellness history

PHASE 4: PERFORMANCE TESTS (Week 4)

Step 4.1: Performance Test Models

Backend:

Create PerformanceTest model

Add fields for all test types from spec

Create /api/performance-tests endpoints

Frontend:

Create PerformanceTestForm component

Add form for basic tests (height, weight, push-ups, etc.)

Display test history table

Deliverable: Coach can record performance tests for athletes

Step 4.2: Performance Graphs & History

Frontend:

Create PerformanceHistory component

Add charts for specific metrics over time

Add comparison to previous tests

Create export to CSV functionality

Deliverable: Visual display of athlete performance trends

PHASE 5: COACH DASHBOARD (Week 5)

Step 5.1: Team Overview Dashboard

Backend:

Create /api/dashboard/team-overview

Aggregate wellness scores by team

Calculate team averages

Add filtering by sport/position

Frontend:

Create CoachDashboard component

Display team wellness overview

Add athlete status cards (red/yellow/green)

Create filtering controls

Deliverable: Coach sees team wellness status at a glance

Step 5.2: Flagging System

Backend:

Add flag calculation logic

Create rules for red/yellow/green flags

Add alert generation endpoint

Frontend:

Add visual flags to athlete cards

Create alerts panel

Add notification system

Deliverable: Automated flagging system for at-risk athletes

PHASE 6: INJURY & CONCUSSION LOGGING (Week 6)

Step 6.1: Injury Tracking

Backend:

Create InjuryRecord model

Add fields: type, severity, date, recovery_status

Create /api/injuries endpoints

Frontend:

Create InjuryLogForm component

Add injury history to athlete profile

Create injury status indicators

Deliverable: Track and log athlete injuries

Step 6.2: Concussion Protocol

Backend:

Extend InjuryRecord for concussions

Add concussion-specific fields

Create clearance workflow

Frontend:

Add concussion-specific form

Create clearance checklist

Add return-to-play timeline

Deliverable: Complete concussion management system

PHASE 7: WORKLOAD & RECOVERY (Week 7)

Step 7.1: Training Load Tracking

Backend:

Create WorkloadSession model

Add RPE, duration, workload calculation

Create /api/workload endpoints

Implement acute:chronic workload ratio

Frontend:

Create WorkloadForm component

Add workload history chart

Display acute:chronic ratios

Deliverable: Track training loads and calculate injury risk metrics

Step 7.2: Recovery & Prehab

Backend:

Create RecoverySession model

Add prehab recommendations system

Create /api/recovery endpoints

Frontend:

Create RecoveryForm component

Add prehab exercise library

Display recovery recommendations

Deliverable: Recovery tracking and prehab assignment

PHASE 8: ADMIN FEATURES (Week 8)

Step 8.1: User Management

Backend:

Add admin role and permissions

Create user management endpoints

Add team management endpoints

Frontend:

Create AdminPanel component

Add user CRUD operations

Add team assignment interface

Deliverable: Admin can manage users and teams

Step 8.2: Data Export & Reports

Backend:

Create CSV/PDF export endpoints

Add data validation endpoints

Create report generation

Frontend:

Add export buttons to dashboards

Create report configuration

Add bulk operations

Deliverable: Export data in multiple formats

PHASE 9: POLISH & MVP COMPLETION (Week 9)

Step 9.1: Mobile Responsiveness

Make all components responsive

Optimize forms for mobile

Add mobile navigation

Test on different screen sizes

Step 9.2: Notifications & Reminders

Add email/SMS reminders

Create notification center

Add proactive alerts

Implement daily check-in reminders

Step 9.3: Testing & Bug Fixes

Add unit tests for critical functions

Perform integration testing

Fix all known bugs

Optimize performance

Step 9.4: Deployment Preparation

Set up PostgreSQL for production

Configure environment variables

Create deployment scripts

Add documentation

IMPLEMENTATION PRIORITIES

MVP CORE (Essential for launch)

✓ User Authentication

✓ Athlete Management

✓ Daily Wellness Check

✓ Coach Dashboard with Flagging

✓ Performance Test Entry

✓ Injury/Concussion Log

MVP NICE-TO-HAVE

✓ Export to CSV/PDF

✓ Prehab Prompts

Mobile Check-in

Testing Calendar

Visual Progression Graphs

STARTING POINT: SIMPLEST VERSION

Let's begin with Phase 1, Step 1.1. I'll create:

Backend Structure:

text

athens-backend/

```
|— app/
|   |— __init__.py
|   |— models.py
|   |— routes/
|   |   |— __init__.py
|   |   |— auth.py
|   |   |— athletes.py
|   |— utils.py
|— migrations/
|— requirements.txt
|— run.py
```

Frontend Structure:

text

athens-frontend/

```
|— public/
|— src/
|   |— components/
|   |   |— Login.jsx
|   |   |— Signup.jsx
|   |   |— Layout.jsx
|   |— services/
|   |   |— api.js
|   |— App.js
|   |— index.js
|— package.json
```